

Documentação Técnica

Rede “Raízes do Nordeste” — Backend API em DDD

Este documento descreve o funcionamento da aplicação, sua arquitetura, modelo de dados, fluxo crítico, endpoints, segurança/LGPD, plano de testes e o ciclo de evolução do projeto (o que foi feito, da versão inicial à atual). A API simula o backend de uma rede de lanchonetes multiunidade com pedidos por múltiplos canais, estoque por unidade, pagamento mock, fidelidade, autorização por perfil e trilha de auditoria.

1. Visão geral

A aplicação é uma API REST construída com **FastAPI** e **SQLAlchemy**, seguindo **Domain-Driven Design (DDD)** com quatro camadas. O objetivo é atender uma rede de lanchonetes em expansão, com matriz e unidades, suportando os canais APP, WEB, TOTEM, BALCÃO e PICKUP. Persistência padrão em SQLite (com opção de PostgreSQL via Docker).

Stack: Python 3.11+, FastAPI, SQLAlchemy 2, Pydantic v2, JWT (python-jose), bcrypt (passlib), Uvicorn, Pytest e Docker.

2. Arquitetura (DDD)

A regra de dependência aponta sempre para o domínio: as camadas externas dependem das internas, nunca o contrário.

```
API -> APPLICATION -> DOMAIN <- INFRASTRUCTURE
(rotas)   (casos de uso)   (regras)   (ORM / repositórios / seed)
```

- **domain/** — entidades ricas (Usuario, Unidade, Produto, EstoqueItem, Pedido, ItemPedido, Pagamento, Fidelidade), value objects (Money em centavos, Email), enums, exceções de negócio e **interfaces de repositório (ports)**.
- **application/** — services (casos de uso): auth, unidade, produto, estoque, pedido, pagamento e fidelidade. Orquestram o domínio e os repositórios.
- **infrastructure/** — modelos ORM, mappers (ORM ■ domínio), repositórios SQLAlchemy que implementam os ports, auditoria e seed.
- **api/** — routers FastAPI, schemas Pydantic (DTOs), presenters, autenticação JWT e autorização por perfil.

Por que separar entidade de domínio do modelo ORM? As entidades carregam as invariantes do negócio (ex.: Pedido.atualizar_status valida a máquina de estados; EstoqueItem.saida impede estoque negativo). Os modelos ORM cuidam só da persistência. Os mappers traduzem entre os dois mundos, mantendo o domínio livre de framework e testável sem banco.

Estrutura de pastas

app/	
main.py	bootstrap FastAPI + lifespan (init DB / seed)
core/	config, security (JWT/bcrypt), exceptions, deps (DI)
domain/	
entities/	Usuario, Unidade, Produto, EstoqueItem, Pedido, ItemPedido, Pagamento, Fidelidade
value_objects/	Money, Email
enums.py	CanalPedido, StatusPedido, StatusPagamento, Role
repositories/	interfaces (ports)
application/services/	auth, unidade, produto, estoque, pedido, pagamento, fidelidade
infrastructure/	database, models (ORM), mappers, repositories/ (impl), seed
api/	routers + presenters
schemas/	DTOs Pydantic (request/response)

3. Modelo de dados (DER lógico)

```
USUARIO 1 ---- N PEDIDO N ---- 1 UNIDADE
PEDIDO  1 ---- N ITEM_PEDIDO N ---- 1 PRODUTO
UNIDADE 1 ---- N ESTOQUE_UNIDADE N ---- 1 PRODUTO   (estoque por unidade)
PEDIDO  1 ---- 1 PAGAMENTO
```

USUARIO 1 ---- 1 FIDELIDADE
AUDITORIA (registro independente de ações sensíveis)

Tabela	Campos principais	Observações
usuarios	id, nome, email(unique), senha_hash, role, consentimento_lgpd	Senha só em hash bcrypt
unidades	id, nome, cidade, ativo	Lojas da rede
produtos	id, nome, preco_cents, ativo	Catálogo (cardápio base)
estoque_unidade	id, unidade_id(FK), produto_id(FK), quantidade	UNIQUE(unidade,produto)
pedidos	id, user_id(FK), unidade_id(FK), canal_pedido, status, created_at	Canal e unidade no pedido
itens_pedido	id, pedido_id(FK), produto_id(FK), quantidade, preco_unitario_cents	Preço congelado no item
pagamentos	id, pedido_id(FK,unique), valor_cents, status	1:1 com pedido
fidelidade	id, user_id(FK,unique), pontos	Saldo de pontos
auditoria	id, acao, entidade, entidade_id, user_id, detalhe, timestamp	Rastreabilidade

Valores monetários são tratados em **centavos (inteiro)** via o Value Object Money, evitando erros de ponto flutuante. O total do pedido é derivado da soma dos itens (não persistido).

4. Fluxo crítico

```
POST /pedidos
  -> valida unidade, produtos e ESTOQUE da unidade; status AGUARDANDO_PAGAMENTO
POST /pagamentos/{pedidoId}
  -> pagamento mock (APROVADO / RECUSADO)
  -> se APROVADO: status PAGO + baixa de estoque da unidade + pontos de fidelidade
PATCH /pedidos/{id}/status -> EM_PREPARO -> PRONTO -> ENTREGUE
POST /pedidos/{id}/cancelar -> CANCELADO (repõe estoque se já estava pago)
```

Máquina de estados do pedido

```
AGUARDANDO_PAGAMENTO --> PAGO --> EM_PREPARO --> PRONTO --> ENTREGUE
      |               |               |               |
      +-----+-----+-----+-----+-----+-----+--> CANCELADO
(transição inválida => HTTP 422 TRANSICAO_STATUS_INVALIDA)
```

A baixa de estoque ocorre **apenas no pagamento aprovado**, evitando reservar estoque de pedidos que nunca serão pagos. O cancelamento de um pedido já pago **repõe** o estoque na unidade.

5. Endpoints e contratos

Método e rota	Auth
POST /auth/register · POST /auth/login	público
GET /unidades · /unidades/{id} · /unidades/{id}/cardapio	público
POST /unidades	GERENTE/ADMIN
GET /produtos?page=&limit= · GET /produtos/{id}	público
POST /produtos	GERENTE/ADMIN
GET /estoque/{unidadeId}	público
POST /estoque/{unidadeId}/movimentar	GERENTE/ADMIN
POST /pedidos	autenticado
GET /pedidos?canalPedido=&status=&unidadeId=&page=&limit=	autenticado
GET /pedidos/{id}	autenticado
PATCH /pedidos/{id}/status	ATENDENTE/COZINHA/GERENTE/ADMIN
POST /pedidos/{id}/cancelar	autenticado
POST /pagamentos/{pedidoId}	autenticado
GET /fidelidade/saldo · POST /fidelidade/resgatar	autenticado
GET /auditoria	GERENTE/ADMIN

Contrato — criar pedido (camelCase)

```
POST /pedidos      Authorization: Bearer <token>
{
  "canalPedido": "TOTEM",
  "unidadeId": 1,
  "itens": [ { "produtoId": 1, "quantidade": 2 } ],
  "formaPagamento": "MOCK"
}
-> 201
{
  "pedidoId": 9001, "clienteId": 4, "unidadeId": 1,
  "canalPedido": "TOTEM", "status": "AGUARDANDO_PAGAMENTO",
  "total": 57.80, "createdAt": "2026-06-23T12:00:00Z",
  "itens": [ { "produtoId": 1, "quantidade": 2, "precoUnitario": 28.90, "subtotal": 57.80 } ]
}
```

Padrão de erro (único para toda a API)

```
{
  "error": "ESTOQUE_INSUFICIENTE",
  "message": "Não há quantidade suficiente para um ou mais itens.",
  "details": [ { "field": "produto_id:1", "issue": "Disponível: 1" } ],
  "timestamp": "2026-06-23T12:00:00Z",
  "path": "/pedidos"
}
```

Status codes: 200, 201, 400/422 (validação), 401 (sem autenticação), 403 (sem permissão), 404 (não encontrado), 409 (conflito/regra de negócio).

6. Segurança, LGPD e auditoria

- **Autenticação** JWT (HS256); o `user_id` vem do token (claim `uid`), não do corpo da requisição.

- **Autorização por perfil** (CLIENTE, ATENDENTE, COZINHA, GERENTE, ADMIN) aplicada nos endpoints; acesso indevido retorna 403.
- **Senhas** apenas em hash bcrypt; senha_hash nunca é exposta nas respostas.
- **LGPD**: coleta mínima (nome e e-mail); consentimento obrigatório no cadastro e para fidelidade; finalidade clara (identificação e login).
- **Auditoria**: criação de pedido, mudança de status, cancelamento e pagamento são registrados na tabela auditoria (consultável por GERENTE/ADMIN em GET /auditoria).

7. Como executar

Docker

```
docker compose up --build      # Swagger em http://127.0.0.1:8000/docs
```

Local

```
python -m venv .venv && source .venv/bin/activate
pip install -r requirements.txt
uvicorn app.main:app --reload
```

Testes

```
pytest -q      # 17 cenários (T01..T17)
```

Seed automático cria 2 unidades, 6 produtos, estoque inicial e usuários:

admin@raizes.com/admin123 (ADMIN), gerente@raizes.com/gerente123 (GERENTE),
atendente@raizes.com/atend123 (ATENDENTE).

8. Plano de testes (executável)

A suíte automatizada (Pytest) e a coleção Postman cobrem 17 cenários (positivos e negativos). Cobertura: autenticação/autorização (401/403), validação (422), regras de negócio (404/409) e fluxo de pagamento.

ID	Cenário	Endpoint	Esperado
T01	Login válido	POST /auth/login	200 + accessToken
T02	Acesso sem token	GET /pedidos	401 NAO_AUTENTICADO
T03	Perfil sem permissão	POST /produtos (CLIENTE)	403 SEM_PERMISSAO
T04	Consentimento LGPD ausente	POST /auth/register	422 CONSENTIMENTO_LGPD_OBRIGATORIO
T05	Credenciais inválidas	POST /auth/login	401 CREDENCIAIS_INVALIDAS
T06	Criar pedido válido	POST /pedidos	201 AGUARDANDO_PAGAMENTO
T07	canalPedido ausente	POST /pedidos	422 VALIDACAO
T08	Unidade inexistente	POST /pedidos	404 UNIDADE_NAO_ENCONTRADA
T09	Estoque insuficiente	POST /pedidos	409 ESTOQUE_INSUFICIENTE
T10	Filtro por canal	GET /pedidos?canalPedido=WEB	200 só WEB
T11	Transição inválida	PATCH /pedidos/{id}/status	422 TRANSICAO_STATUS_INVALIDA
T12	Pagamento aprovado	POST /pagamentos/{id}	201 APROVADO + baixa estoque
T13	Pagamento recusado	POST /pagamentos/{id}	RECUSADO, estoque mantido
T14	Pagamento duplicado	POST /pagamentos/{id}	409 PAGAMENTO_JA_PROCESSADO
T15	Fluxo completo + fidelidade	vários	ENTREGUE + pontos > 0
T16	Cancelar repõe estoque	POST /pedidos/{id}/cancelar	CANCELADO + estoque reposto
T17	Resgate de fidelidade	POST /fidelidade/resgatar	200 saldo - pontos

9. Ciclo do que foi feito (evolução do projeto)

9.1 Ponto de partida (v1)

A primeira versão entregava o esqueleto em DDD com o fluxo principal: cadastro/login com JWT, produtos com estoque global, pedido, pagamento mock, atualização de status e baixa de estoque. Padrão de erro simples (`{error:true, message, code}`), 12 testes e Docker.

9.2 Lacunas identificadas frente ao roteiro

- Sem **autorização por perfil** (403) — exigida no MVP.
- Sem **unidades** nem **estoque por unidade** (a rede é multiunidade).
- Sem **fidelidade**, **cancelamento**, **paginação** e **auditoria**.
- Padrão de erro e nomes de contrato divergentes do roteiro (`camelCase`, `details`, `timestamp`, `path`).
- Plano de testes sem cenário 403 e sem formato T01..Tn.

9.3 O que foi implementado (v2)

- **RBAC**: dependência `requires_roles` e papéis `CLIENTE/ATENDENTE/COZINHA/GERENTE/ADMIN` aplicados nos endpoints (403).
- **Unidades + estoque por unidade**: entidades `Unidade` e `EstoqueItem`; cardápio por unidade; pedido vinculado à unidade; validação e baixa por unidade.
- **Fidelidade** (acúmulo no pagamento e resgate, com consentimento), **cancelamento** com reposição de estoque, **paginação** (`page/limit`) e **created_at** no pedido.
- **Auditoria** de ações sensíveis + endpoint `GET /auditoria`.
- **Padrão de erro** no formato do roteiro (`error`, `message`, `details`, `timestamp`, `path`) + handler de validação 422.
- **Contrato camelCase** (`canalPedido`, `unidadeId`, `produtoId`, `accessToken`...).
- Seed ampliado (unidades, estoque, 3 usuários por papel); **17 testes** (T01..T17); coleção Postman reorganizada em pastas com cenários de erro.

9.4 O que ficou como proposta (justificado)

- **Unit of Work** (transação única no pagamento): hoje cada repositório faz commit; em produção, o ideal é uma transação atômica.
- **Migrations (Alembic)**: usa-se `create_all` + seed; suficiente para o escopo acadêmico.
- **Concorrência de estoque** (bloqueio otimista/pessimista), **refresh token**, **promoções/campanhas** (documentadas como regra) e **anonimização** LGPD ficam como evolução.